

RUNBOOK — V1.0

AutoTestGen System

Step-by-step execution guide for automated
Playwright test project generation from any web
application

DOCUMENT TYPE

Operational Runbook

VERSION

1.0.0

DATE

May 2026

AUTH SUPPORT

OIDC · Ping PKCE · Basic

FRAMEWORKS

Playwright · Selenium · TypeScript · JavaScript · Java

Table of Contents

This runbook covers everything from first-time setup to running the full generation pipeline

PRE	Prerequisites & System Requirements	Node.js, VS Code, MCP, environment setup
1	First-Time Setup	Install dependencies, configure MCP, prepare workspace
2	Configure Your Application	Fill config.json, set environment variables, choose project mode
3	Pipeline Overview	The 8 stages and what each one does
4	Running the Pipeline	Launch master orchestrator, monitor stages, handle human review gates
5	Stage-by-Stage Reference	What each stage does, inputs, outputs, failure handling
6	Authentication Flows	PKCE/Channel Secure Ping, confidential client, basic auth
7	Existing Project Merge Guide	How the system analyzes and merges into an existing test project
8	Running Generated Tests	Execute, filter, report — smoke, regression, e2e
9	Troubleshooting	Common issues and how to resolve them
APP	Appendix — Config Reference & Security Notes	All config.json fields, corporate network guidance



PREREQUISITES

System Requirements

Ensure the following are available on the machine before starting

REQUIREMENT	VERSION	CHECK COMMAND	STATUS
Node.js	18.x or above	<code>node --version</code>	Required
VS Code	1.96 or above	Help → About	Required
GitHub Copilot Extension	Latest	Extensions panel	Required
Playwright MCP	Latest (auto-installed)	<code>npx @playwright/mcp --version</code>	Auto-installed
Git	Any	<code>git --version</code>	Recommended
Internet or internal npm mirror	—	Proxy/Artifactory config	For npm install



Corporate Network Note

The Playwright MCP server runs entirely on your local machine. No application data is sent outside the machine through MCP. The only external calls are to the AI provider API (same boundary as using GitHub Copilot on source code) and to npm for package installation.

Agent Mode Requirement

This system uses **VS Code Agent Mode** (also called Copilot Workspace or Agentic mode). Verify it is enabled:

- 1

Open VS Code Settings

Press `Cmd+,` (macOS) or `Ctrl+,` (Windows/Linux)
- 2

Search for "agent mode"

Ensure `github.copilot.chat.agent.enabled` is checked

3

Enable MCP support

Search `chat.mcp.enabled` and ensure it is set to `true`



SECTION 1

First-Time Setup

One-time steps required before the first pipeline run

Step 1.1 — Open the AutoTestGen Workspace

Open VS Code and open the `AutoTestGen-System` folder as your workspace:

```
File → Open Folder → AutoTestGen-System/
```

Step 1.2 — Install the Playwright MCP Server

Place the MCP configuration file so VS Code can discover and start the Playwright MCP server automatically:

- 1 **Create the `.vscode` folder at your workspace root (if it does not exist)**

```
mkdir -p .vscode
```

- 2 **Copy the MCP config**

Copy `mcp-config/mcp.json` to `.vscode/mcp.json`

- 3 **Reload VS Code window**

Press `Cmd+Shift+P` → "Developer: Reload Window"

- 4 **Verify MCP is active**

In the Copilot Chat panel, you should see a tools icon (⚙️) and "playwright" listed as an available tool

```
# .vscode/mcp.json - contents:
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"],
      "env": { "PLAYWRIGHT_HEADLESS": "false" }
    }
  }
}
```



Corporate npm Registry

If your organisation uses an internal npm mirror (Artifactory, Nexus), configure npm to use it before running the above:

```
npm config set registry https://your-internal-npm-mirror.com
```

Step 1.3 — Create the Output Directory

```
mkdir -p output/auth output/logs
```

Step 1.4 — Gitignore Sensitive Files

If using Git, ensure the following are in `.gitignore` to prevent accidental credential leakage:

```
output/auth/storageState.json
output/auth/
.env
config.json
```



SECTION 2

Configure Your Application

The only manual input required — fill config.json once before running

Step 2.1 — Create config.json

Copy the template and fill in your values:

```
cp config.template.json config.json
```

Step 2.2 — Key Fields to Configure

FIELD	EXAMPLE VALUE	NOTES
<code>project.mode</code>	<code>"existing"</code> or <code>"new"</code>	Determines if system merges or scaffolds
<code>project.existingProjectPath</code>	<code>"/path/to/tests"</code>	Absolute path. Only when mode = existing
<code>project.newProjectTechStack</code>	<code>"playwright-typescript"</code>	Only when mode = new
<code>application.baseUrl</code>	<code>"https://myapp.internal.com"</code>	Full URL including protocol
<code>auth.type</code>	<code>"oidc-ping-pkce"</code>	Use pkce for SPA apps with Channel Secure
<code>auth.clientId</code>	<code>"\$OIDC_CLIENT_ID"</code>	Reference env var — never hardcode
<code>auth.clientSecret</code>	<code>null</code>	Leave null for PKCE. Required for

FIELD	EXAMPLE VALUE	NOTES
		confidential clients only
<code>auth.username</code>	<code>"\$APP_USERNAME"</code>	Reference env var
<code>auth.password</code>	<code>"\$APP_PASSWORD"</code>	Reference env var
<code>auth.postLoginUrlPattern</code>	<code>"/dashboard"</code>	URL fragment visible after successful login
<code>crawl.maxDepth</code>	<code>3</code>	1 = homepage only, 3 = recommended

Step 2.3 — Set Environment Variables

Credentials are NEVER stored in config files. Set them as environment variables before running:

```
# In your terminal session before running the pipeline:
export OIDC_CLIENT_ID="your-ping-client-id"
export APP_USERNAME="test-service-account@company.com"
export APP_PASSWORD="your-secure-password"
export BASE_URL="https://your-app.internal.com"
```

Security Rule — No Hardcoded Credentials

Never put actual usernames, passwords, or client secrets directly in `config.json`. Always use the `$VARIABLE_NAME` reference format. The pipeline enforces this rule and will stop if it detects a non-variable value in a credential field.

Step 2.4 — Validate Configuration

Before running the full pipeline, verify your config is valid by reading it and checking all fields are filled:

```
# Quick sanity check:
node -e "const c = require('./config.json'); console.log('App:', c.applicati"
```




SECTION 3

Pipeline Overview

The 8 stages executed in sequence by the master orchestrator



STAGE 0 Input Collection & Validation

Validates config.json, checks environment variables, confirms app URL is reachable, creates output/ directory

→ `output/pipeline-state.json`



STAGE 1 Project Fingerprinting (existing projects only)

Scans existing project: detects framework, language, folder structure, naming conventions, code style, auth pattern

→ `output/project-fingerprint.json`



STAGE 2 OIDC / Ping Authentication

Opens real browser, performs OIDC/PKCE login flow, saves authenticated session. **Pipeline stops if login fails.**

→ `output/auth/storageState.json`



STAGE 3 DOM Crawl & Application Discovery

Crawls all routes using saved auth state. Captures every form, field, button, table, modal, and conditional logic branch

→ `output/application-map.json`



STAGE 4 Test Strategy Generation **|| HUMAN REVIEW**

Applies QA architect heuristics to produce a complete test plan. **Pipeline pauses here for human review before code is generated.**

→ `output/test-plan.json`



STAGE 5 Test Data Factory Generation

Generates structured test data sets (valid, invalid, boundary, edge case) for every form field discovered

→ `[project]/data/*.data.ts`



STAGE 6 Page Object Model Generation

Generates POM classes for every page. Locator priority: data-testid → aria-label → id → name → CSS

→ `[project]/pages/*.ts`

STAGE 7 Test File Generation

Generates actual Playwright spec files from the approved test plan. Organized by module and type. Safe merge for existing projects.

→ `[project]/tests/**/*spec.ts`

STAGE 8 Coverage Gap Analysis ■ HUMAN REVIEW

Audits generated tests vs application map. Identifies critical, high, medium, and low coverage gaps. Final architect review gate.

→ `output/coverage-gap-report.json` + `output/merge-report.json`



Two Human Review Gates

The pipeline pauses at Stage 4 (test plan review) and Stage 8 (gap report review). At Stage 4, you can add, modify, or remove test cases before any code is generated. Reply **"proceed"** to the agent to continue after each review.



SECTION 4

Running the Pipeline

Step-by-step instructions to launch the master orchestrator in VS Code Agent Mode

Step 4.1 — Set Environment Variables in Your Terminal

Before launching VS Code Agent Mode, ensure credentials are exported in your terminal session:

```
export APP_USERNAME="your-test-account"
export APP_PASSWORD="your-password"
export OIDC_CLIENT_ID="your-ping-client-id"
export BASE_URL="https://your-app.internal.com"
```

Step 4.2 — Open the Master Orchestrator Prompt

1 Open VS Code Explorer

Navigate to `orchestrator/master-orchestrator.prompt.md`

2 Open Copilot Chat in Agent Mode

Press `Cmd+Shift+P` → "GitHub Copilot: Open Chat" → Switch mode to **Agent** using the dropdown in the chat panel

3 Reference the orchestrator prompt

In the chat input, type: `#orchestrator/master-orchestrator.prompt.md` and press Enter, OR use the "Run Prompt" button that appears at the top of the `.prompt.md` file when opened

4 Confirm the agent starts Stage 0

The agent will read `config.json`, validate fields, and log "Stage 0: PASSED" before proceeding



Alternate: Run Prompt File Directly

With VS Code 1.96+, you can click the ► "Run in Agent Mode" button that appears in the editor toolbar when a `.prompt.md` file is open. This is the simplest way to trigger the pipeline.

Step 4.3 — Monitor Stage Progress

The agent logs progress to the chat panel. Watch for these stage completion messages:

```
=== STAGE 0 COMPLETE === ✓ Config validated
=== STAGE 1 COMPLETE === ✓ Project fingerprinted
=== STAGE 2 COMPLETE === ✓ Authenticated – storageState saved
=== STAGE 3 COMPLETE === ✓ 12 pages, 8 forms, 3 tables discovered
=== STAGE 4 COMPLETE === *** HUMAN REVIEW REQUIRED ***
```

Step 4.4 — Human Review at Stage 4 (Test Plan)

- 1 Open output/test-plan.json**
Review the generated test cases. Each has an ID, description, steps, expected result, and data requirements.
- 2 Modify if needed**
Set `"approved": false` on any test case to exclude it. Add new entries following the same schema. Edit descriptions to match your terminology.
- 3 Reply "proceed" in the agent chat**
Type `proceed` in the Copilot chat panel. The pipeline continues with Stages 5–8.

Step 4.5 — Human Review at Stage 8 (Gap Report)

After all test files are generated, open `output/coverage-gap-report.json` :

GAP SEVERITY	ACTION REQUIRED
Critical	Must address before merging to main branch. Re-run generation for these areas.
High	Should address before next release cycle.
Medium	Improve coverage over time. Track in backlog.
Low	Nice to have. Address in future iterations.



SECTION 6

Authentication Flows

Configuration for each supported auth type

PKCE / Channel Secure Ping (SPA Applications)

Use this for Single Page Applications using Channel Secure Ping with PKCE. **No client secret required.**

```
"auth": {
  "type": "oidc-ping-pkce",
  "oidcIssuerUrl": "https://ping.your-company.com/as",
  "clientId": "$OIDC_CLIENT_ID",
  "clientSecret": null,          // ← REQUIRED to be null for PKCE
  "pkce": {
    "enabled": true,
    "codeChallengeMethod": "S256",
    "redirectUri": "https://your-app.com/callback"
  },
  "scopes": "openid profile email",
  "username": "$APP_USERNAME",
  "password": "$APP_PASSWORD"
}
```

How PKCE works with the agent

The SPA generates the `code_verifier` and `code_challenge` automatically when the login flow starts. The agent simply fills in the credentials on the Ping login page and follows the redirect — the PKCE token exchange is handled transparently by the browser. The agent does not need to generate or manage the PKCE values manually.

Confidential Client (Server-Side Apps)

```
"auth": {
  "type": "oidc-ping",
  "clientId": "$OIDC_CLIENT_ID",
  "clientSecret": "$OIDC_CLIENT_SECRET",  // ← Required for confidential
  "pkce": { "enabled": false }
}
```

Test Account Requirements

REQUIREMENT	WHY
Dedicated test/service account	Isolate test activity from real users
MFA disabled on the account	Pipeline cannot handle interactive MFA challenges
Minimum required permissions	Principle of least privilege
Password does not expire frequently	Avoid pipeline breakages from expired passwords
Credentials stored in env vars only	Never in config files or source control



SECTION 7

Existing Project Merge Guide

How the system safely merges generated code into an existing test project

Merge Rules (Non-Destructive Always)

SCENARIO	ACTION
New module not yet in project	CREATE new spec file in correct folder
Existing module with new tests	APPEND new tests inside existing describe block
Existing POM — new locators needed	EXTEND — append only new locators/methods
Existing POM — locator already exists	SKIP — never change existing locators
Test with same ID already exists	SKIP + LOG in merge-report.json
Any existing file content	NEVER OVERWRITE — read-only to existing content

After Merge — Review the Merge Report

Open `output/merge-report.json` to see exactly what was created, extended, and skipped:

```
{
  "summary": {
    "filesCreated": 8,
    "filesExtended": 3,
    "filesSkipped": 1,
    "testsAdded": 47,
    "testsDuplicated": 2
  },
  "conflicts": []
}
```

**Commit Before Running**

Always commit or stash your existing project changes before running the pipeline in merge mode. This gives you a clean baseline to diff against and makes it easy to review exactly what was generated.



SECTION 8

Running Generated Tests

Commands to execute the generated test suite

Prerequisites Before Running

- 1 Install dependencies in the generated/existing project

```
cd /path/to/your/test-project && npm install
```

- 2 Install Playwright browsers (first time only)

```
npx playwright install chromium
```

- 3 Set BASE_URL environment variable

```
export BASE_URL="https://your-app.internal.com"
```

- 4 Ensure storageState.json is present

It should exist at `output/auth/storageState.json` from the pipeline run. If expired, re-run Stage 2 only.

Execution Commands

```
# Run all tests
npx playwright test

# Run smoke tests only (quick sanity check)
npx playwright test --grep @smoke

# Run regression tests for a specific module
npx playwright test tests/regression/requests/

# Run a single test by name
npx playwright test --grep "TC-REQ-001"

# Run with visible browser (useful for debugging)
npx playwright test --headed

# Run and open HTML report afterward
npx playwright test && npx playwright show-report

# Run in CI mode (headless, no retries)
CI=true npx playwright test
```

Refreshing Auth State (When Session Expires)

If tests start failing with login redirects, the stored session has expired. Re-run only Stage 2:

```
# Open the Stage 2 prompt directly in Agent Mode:  
# prompts/02-oidc-auth.prompt.md → Run in Agent Mode  
# This re-authenticates and overwrites storageState.json
```



SECTION 9

Troubleshooting

Common issues and resolution steps

SYMPTOM	LIKELY CAUSE	RESOLUTION
Stage 0 fails — "environment variable not set"	Credentials not exported before running	Run <code>export APP_USERNAME=...</code> in the same terminal session and retry
Stage 2 fails — authentication error	Wrong credentials, expired password, or MFA enabled	Verify credentials manually in browser first. Disable MFA on test account in Ping.
Stage 2 fails — SSL error on HTTPS	Corporate proxy intercepting HTTPS (MITM)	Add corporate CA certificate to the Playwright browser: set <code>NODE_EXTRA_CA_CERTS=/path/to/corp-ca.crt</code>
Stage 2 fails — PKCE redirect_uri mismatch	<code>auth.pkce.redirectUri</code> does not match Ping registration	Ask your Ping admin for the exact registered redirect URI and update config.json
MCP browser tool not available	<code>.vscode/mcp.json</code> missing or VS Code not reloaded	Verify <code>.vscode/mcp.json</code> exists. Reload VS Code. Check MCP is enabled in settings.
Stage 3 — crawl misses pages	Pages load dynamically or require user action to reach	Add specific routes to <code>crawl.includeRoutes</code> in config.json. Increase <code>crawl.maxDepth</code> .
Generated tests fail — locator not found	DOM snapshot was stale or app updated since crawl	Re-run Stage 3 to refresh the application map, then re-run Stages 6–7 only.
npm install blocked on	npm registry is firewalled	Configure npm to use internal mirror: <code>npm config set registry https://your-nexus.com/npm</code>

SYMPTOM	LIKELY CAUSE	RESOLUTION
corporate network		
Tests fail after session token expires	storageState.json contains expired OIDC token	Re-run Stage 2 only to refresh the saved session
Merge report shows many "skipped" tests	Test IDs in test-plan.json already exist in project	Normal behaviour — the system never duplicates. Review skipped list in merge-report.json.

Re-Running a Single Stage

Each stage prompt can be run independently in Agent Mode without re-running the full pipeline:

```
# Open any stage prompt file and run it directly:
prompts/02-oidc-auth.prompt.md      # Re-authenticate only
prompts/03-dom-crawl.prompt.md      # Re-crawl if app changed
prompts/07-test-generator.prompt.md  # Re-generate tests after editing test-p
prompts/08-gap-analyzer.prompt.md    # Re-analyze coverage
```



APPENDIX

File Structure Reference

Complete layout of the AutoTestGen-System workspace

```
AutoTestGen-System/ |— config.template.json ← copy to config.json and
fill in |— .vscode/ |   |— mcp.json ← copy from mcp-config/mcp.json |—
orchestrator/ |   |— master-orchestrator.prompt.md ← RUN THIS to start
the pipeline |— prompts/ |   |— 00-input-collection.prompt.md |   |— 01-
project-fingerprint.prompt.md |   |— 02-oidc-auth.prompt.md |   |— 03-dom-
crawl.prompt.md |   |— 04-test-strategy.prompt.md |   |— 05-data-
factory.prompt.md |   |— 06-pom-generator.prompt.md |   |— 07-test-
generator.prompt.md |   |— 08-gap-analyzer.prompt.md |— schemas/ |   |—
application-map.schema.json |   |— project-fingerprint.schema.json |   |—
test-plan.schema.json |— templates/ |   |— playwright-typescript/ ←
scaffold for new TS projects |   |— playwright-javascript/ ← scaffold for
new JS projects |   |— playwright-selenium-dual/ ← scaffold for dual PW +
Selenium |— mcp-config/ |   |— mcp.json ← copy to .vscode/mcp.json |—
output/ ← generated by pipeline (gitignored) |— auth/ |   |—
storageState.json ← SENSITIVE – never commit |— pipeline-state.json |—
application-map.json |— test-plan.json ← REVIEW THIS at Stage 4 |—
coverage-gap-report.json ← REVIEW THIS at Stage 8 |— merge-report.json
← existing project only
```

Quick Start Checklist

- ☐ Node.js 18+ installed
- ☐ VS Code with GitHub Copilot extension installed
- ☐ Agent Mode enabled in VS Code settings
- ☐ MCP support enabled (`chat.mcp.enabled: true`)
- ☐ `mcp-config/mcp.json` copied to `.vscode/mcp.json`
- ☐ VS Code reloaded after MCP config placed
- ☐ `config.template.json` copied to `config.json`
- ☐ All fields in `config.json` filled (no placeholder text remaining)
- ☐ Credential environment variables exported in terminal session
- ☐ `output/auth/` in `.gitignore`

☐ Test service account created in Ping with MFA disabled

☐ (Existing projects) Existing test project committed/stashed before merge run